

Accès API

Le RCDR offre un accès programmatique aux métadonnées de repérage via les API des Collections et de l'infrastructure de Canadiana. Combinez ces points d'accès avec les paramètres de recherche, les facettes, la pagination et le tri.

SUR CETTE PAGE

Points d'accès API du catalogue

Paramètres de requête du catalogue

Structure des réponses du catalogue

Exemples du catalogue

Aperçu IIIF

API Image IIIF

API Presentation IIIF (manifeste)

API Content Search IIIF

Points d'accès API du catalogue

Utilisez ces points d'accès du catalogue pour les recherches, les notices et les facettes. Le paramètre `format` est l'option la plus fiable lorsque les identifiants contiennent des points (par exemple `oocihm.N_00591`).

Points d'accès API du catalogue

POINT D'ACCÈS	MÉTHODE ET CHEMIN
Recherche	GET /catalog?search_field=all_fields&q=&format=json
Notice individuelle	GET /catalog/:id?format=json
Facette	GET /catalog/facet/:facet_id?search_field=all_fields&q=&format=json
Export de notice	GET /catalog/:id?format=marc marcxml xml

Paramètres de requête du catalogue

Paramètres de requête du catalogue

PARAMÈTRE	TYPE	DESCRIPTION	VALEURS / DÉFAUT
q	chaîne	Texte de recherche.	Défaut : chaîne vide
search_field	enum	Champ ciblé pour l'exécution de la requête.	all_fields, full_title_tsim, author_tsim, subject_tsim, tx_gen
lang	enum	Langue de la réponse.	en, fr
page	entier	Numéro de page des résultats (commence à 1).	Défaut : 1
per_page	entier	Nombre de résultats par page.	10, 20, 50, 100
sort	enum	Clé de tri des résultats.	relevance, year-desc, year-asc, date-added-desc, date-added-asc
f[field_name][]=value	tableau	Filtres de facettes pour restreindre les résultats.	pub_date_ssim, language_ssim_str, depositor_tsim_str, subject_ssim_str, author_ssim_str, collectionen_path, collectionfr_path, serial_title_str, is_issue_str, is_serial_str. Exemple : f[language_ssim_str][]=English
range[pub_date_ssim][begin] range[pub_date_ssim][end]	entier	Filtre par intervalle d'années de publication.	Exemple : 1900 à 1910

format	enum	Format de sortie de la réponse.	Recherche : <code>html</code> , <code>json</code> ; Notice individuelle : <code>html</code> , <code>json</code> , <code>marc</code> , <code>marcxml</code> , <code>xml</code>
---------------	------	---------------------------------	--

Structure des réponses du catalogue

Une requête de recherche comme
`/catalog?search_field=all_fields&q=&format=json` retourne:

Champs de structure des réponses du catalogue

CHAMP	DESCRIPTION
links	URLs de navigation (<code>self</code> , <code>next</code> , <code>last</code>).
meta.pages	État de pagination et volumes (<code>current_page</code> , <code>total_pages</code> , <code>limit_value</code> , <code>total_count</code>).
data[]	Notices de résultat avec <code>id</code> , <code>type</code> , <code>attributes</code> et <code>links.self</code> .
included[]	Objets API réutilisables pour les facettes, les champs de recherche et les tris.

```
ResponseJSON{
  "links": {"self": "...", "next": "...", "last": "..."},
  "meta": {
    "pages": {
      "current_page": 1,
      "total_pages": 246,
      "limit_value": 10,
      "total_count": 2454
    }
  },
  "data": [
    {
      "id": "oocihm.N_00591",
      "type": "Newspaper",
      "attributes": {
        "title": "Keneder adler",
        "published_ssm": {
          "type": "document_value",
          "attributes": {"label": "Published", "value":
```

```

"..."},
    },
    "language_ssim": {
        "type": "document_value",
        "attributes": {"label": "Language", "value":
"..."},
    }
},
"links": {"self":
"http://localhost:3000/catalog/oocihm.N\_00591?lang=en"}
},
],
"included": [
    {"type": "facet", "id": "language_ssim_str"},
    {"type": "search_field", "id": "all_fields"},
    {"type": "sort", "id": "date-added-desc"}
]
}

```

Plusieurs valeurs `attributes.*.attributes.value` sont des fragments HTML destinés à l’affichage. Pour un pipeline de données pur, prévoyez de nettoyer ou de filtrer ce balisage.

Exemples du catalogue

Exemple 1 : Réponse JSON catalogue par défaut

Récupérez la réponse catalogue par défaut et lisez le total des résultats.

Codepython`import requests`

`BASE_URL = "http://localhost:3000"`

```

response = requests.get(
    f"{BASE_URL}/catalog",
    params={"search_field": "all_fields", "q": "", "format":
"json"},
    timeout=30,
)
response.raise_for_status()
catalogue = response.json()
print(catalogue["meta"]["pages"]["total_count"])

```

Exemple 2 : Requête avec tri et pagination

Exécutez une recherche par mot-clé avec tri, taille de page et numéro de page.

Codepython`import requests`

`BASE_URL = "http://localhost:3000"`

```
response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "cartes",
        "search_field": "all_fields",
        "sort": "date-added-desc",
        "per_page": 20,
        "page": 1,
        "format": "json",
    },
    timeout=30,
)
response.raise_for_status()
cartes = response.json()
```

Exemple 3 : Filtres de facettes

Appliquez des facettes pour limiter les résultats par langue et chemin de collection.

Codepython`import requests`

`BASE_URL = "http://localhost:3000"`

```
anglais_response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "f[language_ssim_str][]": "English",
        "format": "json",
    },
    timeout=30,
)
anglais_response.raise_for_status()
anglais = anglais_response.json()
```

```

journaux_response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "f[collectionfr_path][]": "Publications en
se%CC%81rie/Journaux",
        "format": "json",
    },
    timeout=30,
)
journaux_response.raise_for_status()
journaux = journaux_response.json()

```

Exemple 4 : Intervalle d'années de publication

Filtrez les notices par plage d'années avec le champ de facette de date.

Codepython`import requests`

```

BASE_URL = "http://localhost:3000"

response = requests.get(
    f"{BASE_URL}/catalog",
    params={
        "q": "",
        "search_field": "all_fields",
        "range[pub_date_ssim][begin]": 1900,
        "range[pub_date_ssim][end]": 1910,
        "format": "json",
    },
    timeout=30,
)
response.raise_for_status()
periode = response.json()

```

Exemple 5 : Notice individuelle en plusieurs formats

Récupérez la même notice en JSON, MARC et MARCXML depuis un seul point d'accès.

Codepython`import requests`

```
BASE_URL = "http://localhost:3000"

json_response = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "json"},
    timeout=30,
)
json_response.raise_for_status()
notice_json = json_response.json()

marc_response = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "marc"},
    timeout=30,
)
marc_response.raise_for_status()
notice_marc = marc_response.text

marcxml_response = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "marcxml"},
    timeout=30,
)
marcxml_response.raise_for_status()
notice_marcxml = marcxml_response.text
```

API IIIF

Points d'accès API IIIF

IIIF est un cadre commun pour publier, lier et réutiliser des objets du patrimoine numérique. Les ressources IIIF sont publiées comme données ouvertes liées avec [JSON-LD](#), qui est une sérialisation JSON de [RDF](#). Cela permet aux clients de suivre des identifiants stables entre manifestes, canevas, annotations et services d'image.

Les points d'accès sont regroupés dans les sections Image, Manifeste et Content Search ci-dessous.

Documentation officielle des API IIIF: iiif.io/api.

API Image IIIF

Points d'accès API Image IIIF

POINT D'ACCES	CHEMIN
Point d'accès info image	GET {image_service}/info.json
Point d'accès rendu image	GET {image_service}/{region}/{size}/{rotation}/{quality}.{format}

Spécification complète: [IIIF Image API 2.1](#).

Formats de réponse

Formats de réponse API Image IIIF

POINT D'ACCES	FORMAT DE RÉPONSE
Point d'accès info image (info.json)	JSON (IIIF Image API)
Point d'accès rendu image	Octets binaires (par exemple image/jpeg)

Paramètres de requête API Image IIIF

PARAMÈTRE	TYPE	DESCRIPTION	VALEURS / EXEMPLES
region	chaîne	Région de l'image à extraire.	full , x,y,w,h, pct:x,y,w,h
size	chaîne	Dimensions de sortie visées.	full , w,, ,h, w,h, pct:n
rotation	nombre	Angle de rotation de l'image produite.	0, 90, 180, 270
quality	enum	Profil de rendu pris en charge par le service.	default , color, gray, bitonal
format	enum	Format binaire retourné par le point d'accès.	jpg

Forme de la réponse API Image IIIF

info.json retourne des métadonnées JSON:


```

ResponseJSON{
  "@context": "http://iiif.io/api/image/2/context.json",
  "@id": "{image_service}",
  "protocol": "http://iiif.io/api/image",
  "width": 3936,
  "height": 3200,
  "sizes": [{"width": 123, "height": 100}, "..."],
  "tiles": [{"width": 512, "height": 512, "scaleFactors": [1, 2,
4, 8, 16, 32]}],
  "profile": [
    "http://iiif.io/api/image/2/level1.json",
    {
      "formats": ["jpg"],
      "qualities": ["bitonal", "default", "gray", "color"],
      "supports": ["regionByPx", "sizeByW", "rotationBy90s",
"..."]
    }
  ]
}

```

Exemples API Image IIIF

Exemple : Requêtes image IIIF

Interrogez le service d'image pour lire les capacités et générer des dérivés courants.

Codepython`import requests`

```

def url_image_iiif(
    service_id,
    region="full",
    size="full",
    rotation="0",
    quality="default",
    fmt="jpg",
):
    return
f"{service_id}/{region}/{size}/{rotation}/{quality}.{fmt}"

```

```

service_id = "https://image-
tor.canadiana.ca/iiif/2/69429%2Fc2025000026xr85"

```

```

# Découvrir les capacités
info = requests.get(f"{service_id}/info.json", timeout=30).json()
print(info["width"], info["height"]) # 3936 3200

# Image complète redimensionnée à 500 px de large
img_small = requests.get(url_image_iiif(service_id, size="500,"),
    timeout=30)
print(img_small.headers["Content-Type"]) # image/jpeg

# Recadrage par région en pourcentage, puis redimensionnement
img_crop = requests.get(
    url_image_iiif(service_id, region="pct:25,25,50,50",
    size="800,"),
    timeout=30,
)

# Sortie avec rotation
img_rotated = requests.get(url_image_iiif(service_id, size="500,",
    rotation="90"), timeout=30)

# Qualité alternative
img_gray = requests.get(url_image_iiif(service_id, size="500,",
    quality="gray"), timeout=30)

```

Exemple : Réponse image binaire

Vérifiez le type MIME retourné, puis utilisez le contenu binaire de l'image.

Codepythonimport requests

```

response = requests.get(
    "https://image-
    tor.canadiana.ca/iiif/2/69429%2Fc2025000026xr85/full/500,/0/default.
    jpg",
    timeout=30,
)
response.raise_for_status()

print(response.headers["Content-Type"]) # image/jpeg
jpeg_binaire = response.content          # bytes
print(len(jpeg_binaire))

```

API Presentation IIIF (manifeste)

L'exemple ci-dessous est basé sur le manifeste `m2025000007606h` (collection `69429`). Il s'agit d'un manifeste IIIF Presentation 3 avec 842 canevas.

Point d'accès manifeste IIIF

POINT D'ACCES	CHEMIN
Point d'accès manifeste	GET {presentation_service}/69429/m2025000007606h

Spécification complète: [IIIF Presentation API 3.0](#).

Formats de réponse

Formats de réponse du manifeste IIIF

POINT D'ACCES	FORMAT DE RÉPONSE
Point d'accès manifeste	JSON (IIIF Presentation 3)

Forme de la réponse manifeste (Presentation 3)

ResponseJSON{

```
"@context": "http://iiif.io/api/presentation/3/context.json",
"id": "{presentation_service}/69429/m2025000007606h",
"type": "Manifest",
"label": {"en": ["Census returns for the 1901 Canadian census"]},
"metadata": [
  {
    "label": {"en": ["Slug"]},
    "value": {"en": ["oocihm.lac_reel_t6428"]}
  }
],
"summary": {"en": ["..."]},
"items": [
  {
    "id":
"{presentation_service}/canvas/69429/c2025000026xr85",
    "type": "Canvas",
    "width": 3936,
    "height": 3200,
    "label": {"en": ["Image 1"]},
```

```

    "items": [
      {
        "type": "AnnotationPage",
        "items": [
          {
            "type": "Annotation",
            "motivation": "painting",
            "target":
"{presentation_service}/canvas/69429/c2025000026xr85",
            "body": {
              "id":
"{image_service}/full/max/0/default.jpg",
              "type": "Image",
              "format": "image/jpeg",
              "width": 3936,
              "height": 3200,
              "service": [
                {
                  "id": "{image_service}",
                  "type": "ImageService2",
                  "profile": "level2"
                }
              ]
            }
          }
        ]
      }
    ],
    "thumbnail": [{"id":
"{image_service}/full/200,/0/default.jpg", "type": "Image",
"format": "image/jpeg"}]
  }
]
}

```

API Content Search IIIF

Point d'accès Content Search IIIF

POINT D'ACCES	CHEMIN

Point d'accès Content Search (gabarit)	GET /search/{repository_id}/{manifest_id}?q={term}
Point d'accès Content Search (exemple)	GET https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j?q=roi

Spécification complète: [IIIF Content Search API 2.0](#).

Formats de réponse

Formats de réponse Content Search IIIF

POINT D'ACCES	FORMAT DE RÉPONSE
Point d'accès Content Search	JSON-LD (IIIF Search API 2.0); le type racine est AnnotationPage , items contient les annotations de correspondance, et next est présent en cas de pagination.

Forme de réponse (IIIF Search API 2.0)

Champs de forme de réponse Content Search IIIF

CHAMP	DESCRIPTION
@context	Contexte JSON-LD (par exemple http://iiif.io/api/search/2/context.json).
id	URL du point d'accès Content Search pour le manifeste visé.
type	AnnotationPage .
items[]	Tableau d'annotations de correspondance; chaque élément inclut motivation , body.value , et target avec un sélecteur #xywh= .
next	URL de pagination optionnelle (par exemple ?start=100&q=roi).

```
ResponseJSON{
  "@context": "http://iiif.io/api/search/2/context.json",
  "id": "https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j",
```

```

    "type": "AnnotationPage",
    "items": [
        {
            "id": "https://crkn-iiif-api.azurewebsites.net/manifest/69429/m0tt4fn16j1j/canvas/69429/c05t3g16rb0q/text/at/667%2C253%2C100%2C110",
            "type": "Annotation",
            "motivation": "highlighting",
            "body": {
                "type": "TextualBody",
                "value": "Roi",
                "format": "text/plain"
            },
            "target": "https://crkn-iiif-api.azurewebsites.net/manifest/69429/m0tt4fn16j1j/canvas/69429/c05t3g16rb0q#xywh=667,253,100,110"
        }
    ],
    "next": "https://crkn-iiif-content-search.azurewebsites.net/search/69429/m0tt4fn16j1j?start=100&q=roi"
}

```

Exemple : ARK vers manifeste et Content Search

Extrayez l'ARK à partir des métadonnées du catalogue, construisez le chemin IIIF, récupérez le manifeste, puis lancez Content Search.

```

Codepythonimport re
import requests

```

```

BASE_URL = "http://localhost:3000"
PRESENTATION_SERVICE = "{presentation_service}"
CONTENT_SEARCH_SERVICE = "{content_search_service}"

```

```

# 1) Récupérer une notice et extraire l'ARK (href) depuis la valeur HTML

```

```

notice = requests.get(
    f"{BASE_URL}/catalog/oocihm.N_00591",
    params={"format": "json"},
    timeout=30,
).json()
ark_html =
notice["data"]["attributes"]["ark"]["attributes"]["value"]

```

```

ark_url = re.search(r'href="([^\"]+)"', ark_html).group(1)

# 2) Normaliser le chemin ARK pour les services IIIF
ark_path = re.sub(r"^https?://n2t\\.net/ark:/?|^ark:/?", "",
ark_url).rstrip("/")
# "69429/m2025000007606h"

# 3) Récupérer le manifeste
manifeste = requests.get(f"{PRESENTATION_SERVICE}/{ark_path}",
timeout=30).json()
print(manifeste["type"], len(manifeste["items"])) # Manifest 842

# 4) Récupérer le service image du premier canevas
service_image =
manifeste["items"][0]["items"][0]["items"][0]["body"]["service"][0][
"id"]
info = requests.get(f"{service_image}/info.json", timeout=30).json()
print(info["width"], info["height"])

# 5) Exécuter une recherche IIIF Content Search
recherche = requests.get(
    f"{CONTENT_SEARCH_SERVICE}/{ark_path}",
    params={"q": "canada"},
    timeout=30,
).json()
print(recherche["type"], len(recherche["items"]))

```

Pour un aperçu plus général des concepts et outils IIIF, consultez [Qu'est-ce que IIIF?](#).